Báo cáo kết quả huấn luyện & Thử nghiệm

Đề tài: Phát hiện trùng lặp mã nguồn C/C++ và Python

Ngày 17 tháng 10 năm 2026

1 Kết quả tuần này

- Tạo bộ test thay đổi vòng lặp `for`/`while`/`do-while`, đổi IO và tên biến, thêm cặp non-clone khó.
- 12-10: chạy thêm M1-512, M2-512 (warm-start từ checkpoint 256, `max_length` 512, 2 epoch).
- 15-10: chạy thêm M5-512 (Hybrid trên nền encoder 512, 2 epoch).
- 17-10: chuyển demo và mục minh hoạ sang bản 512 cho M1/M2/M5 (ngưỡng lấy theo `threshold.json` của run 512), chấm lại bộ test `custom_suite`.

2 Kết quả huấn luyện

F1 trên tập val theo từng epoch, ngưỡng chọn bằng val.

Epoch	M3	M4	M1-256	M2-256	M1-512	M2-512	M5-256	M5-512
E0	0,789	0,782	0,919	**0,915**	**0,921**	0,932	**0,914**	**0,915**
E1	0,814	0,773	**0,919**	0,914	0,918	**0,933**	0,893	0,900
E2	0,841	0,806	0,912	0,911	—	—	0,859	—
E3	**0,853**	0,794	—	—	—	—	0,879	—
E4	0,804	0,833	—	—	—	—	—	—
E5	0,823	0,816	—	—	—	—	—	—
E6	0,841	0,804	—	—	—	—	—	—
E7	—	0,817	—	—	—	—	—	—
E8	—	0,816	—	—	—	—	—	—
E9	—	0,836	—	—	—	—	—	—
E10	—	0,827	—	—	—	—	—	—
E11	—	0,830	—	—	—	—	—	—
E12	—	0,837	—	—	—	—	—	—
E13	—	0,838	—	—	—	—	—	—
E14	—	**0,842**	—	—	—	—	—	—

Trong đó: M1 = CodeBERT, M2 = GraphCodeBERT, M3 = ASTNN, M4 = CPG-GNN, M5 = Hybrid.

3 Kết quả test

PP	Prec.	Rec.	F1	AUC	Ngưỡng
M3	0,760	0,894	0,821	0,889	0,420
M4	0,723	0,887	0,797	0,882	0,470
M1-256	0,931	0,890	0,910	0,973	0,590
M2-256	0,883	0,933	0,907	0,971	0,430
M1-512	0,947	0,935	**0,941**	**0,987**	0,210
M2-512	0,951	0,959	**0,955**	**0,991**	0,660
M5-256	0,905	0,922	**0,913**	0,970	0,370
M5-512	0,961	0,942	**0,952**	0,988	0,010

3.1 So sánh 256 và 512 token

Lên 512 token thì cả ba đều tăng điểm:

- M1: 0,910 → 0,941.
- M2: 0,907 → 0,955.
- M5: 0,913 → 0,952.

4 Minh hoạ

Minh hoạ dùng bộ test `data/custom_suite` (26 cặp). M1, M2 và M5 chấm bằng bản 512 — bản đã chốt dùng cho demo và báo cáo; M3 và M4 không có biến thể 512 (một cái làm việc trên cây AST, một cái trên node CPG) nên giữ nguyên bản gốc.

Cách đọc hình:

- Hai phía cùng màu là hai đoạn mà model cho là khớp nhau.
- Mỗi model có một bảng màu riêng.
- CodeBERT, GraphCodeBERT, Hybrid ghép theo từng từ (vector encoder).
- ASTNN ghép theo câu lệnh (vector RvNN); bỏ các span chiếm hơn 60% file và chỉ giữ statement cùng cấp, không bôi cả hàm.
- CPG-GNN ghép theo dòng (vector node GAT).
- MOSS tô đúng vùng khớp mà server Stanford trả về; cặp nào không được gắn cờ thì để trắng.
- Ô vàng lẻ là token model dựa vào để cho điểm nhưng không ghép được cặp, phần không tô màu là còn lại.

4.1 Cặp clone khó: P1_combo

Cùng bài tính tổng 1..n, nhưng bản B viết lại hết: `for` đổi thành `while`, `cin/cout` đổi thành `scanf/printf`, đổi tên biến, đảo điều kiện, thêm biến thừa. Cả 5 model đều cho là clone (M1 1,000, M2 1,000, M3 0,812, M4 0,915, M5 1,000), đúng cả. Riêng MOSS cho 0,000 nên bỏ sót cặp này, do server chỉ trả top match nên mấy cặp Type-3/4 viết lại nhiều không được gắn cờ.

CodeBERT — 1,000 (clone)

File A (p1_base_sum)

```
#include <bits/stdc++.h>
using namespace std;
int main () {
int n, sum = 0;
cin >> n ;
for (int i = 1; i <= n; i++) {
sum += i ;
}
cout << sum << endl ;
return 0 ;
}
```

File B (p1_v_combo)

```
#include <cstdio>
int main () {
int N;
scanf("%d", & N );
int acc = 0;
int step = 1;
int unused = 999;
( void) unused;
int k = 1 ;
while (!(k > N)) {
acc = acc + k ;
k = k + step;
}
printf("%d\n", acc );
return 0;
}
```

GraphCodeBERT — 1,000 (clone)

File A (p1_base_sum)

```
#include <bits/stdc++.h>
using namespace std ;
int main () {
int n , sum = 0 ;
cin >> n ;
for ( int i = 1 ; i <= n; i ++) {
sum += i ;
}
cout << sum << endl ;
return 0;
}
```

File B (p1_v_combo)

```
#include <cstdio>
int main () {
int N;
scanf ("%d", & N );
int acc = 0;
int step = 1 ;
int unused = 999 ;
( void )unused;
int k = 1 ;
while (!(k > N )) {
acc = acc + k ;
k = k + step ;
}
printf ( "%d\n" , acc );
return 0;
}
```

ASTNN — 0,812 (clone)

File A (p1_base_sum)

```
#include <bits/stdc++.h>
using namespace std;
int main() {
int n, sum = 0 ;
cin >> n;
for (int i = 1; i <= n; i++) {
sum += i;
}
cout << sum << endl;
return 0;
}
```

File B (p1_v_combo)

```
#include <cstdio>
int main() {
int N;
scanf("%d", &N);
int acc = 0 ;
int step = 1;
int unused = 999;
(void)unused;
int k = 1;
while (!(k > N)) {
acc = acc + k;
k = k + step;
}
printf("%d\n", acc);
return 0;
}
```

CPG-GNN — 0,915 (clone)

File A (p1_base_sum)

```
#include <bits/stdc++.h>
using namespace std;
int main() {
 int n, sum = 0;
 cin >> n;
 for (int i = 1; i <= n; i++) {
 sum += i;
}
cout << sum << endl;
 return 0;
}
```

File B (p1_v_combo)

```
#include <cstdio>
int main() {
 int N;
 scanf("%d", &N);
int acc = 0;
 int step = 1;
int unused = 999;
(void)unused;
int k = 1;
 while (!(k > N)) {
 acc = acc + k;
 k = k + step;
}
printf("%d\n", acc);
 return 0;
}
```

Hybrid — 1,000 (clone)

File A (p1_base_sum)

```
#include <bits/stdc++.h>
using namespace std;
int main () {
int n, sum = 0 ;
cin >> n;
for ( int i = 1 ; i <= n ; i ++) {
sum += i ;
}
cout << sum << endl ;
return 0;
}
```

File B (p1_v_combo)

```
#include <cstdio>
int main () {
int N ;
scanf ("%d", & N );
int acc = 0;
int step = 1 ;
int unused = 999 ;
( void )unused;
int k = 1 ;
while (!( k > N )) {
acc = acc + k ;
k = k + step ;
}
printf ("%d\n", acc );
return 0;
}
```

MOSS (server Stanford) — 0,000 (không clone)

File A

```
#include <bits/stdc++.h>
using namespace std;
int main() {
int n, sum = 0;
cin >> n;
for (int i = 1; i <= n; i++) {
sum += i;
}
cout << sum << endl;
return 0;
}
```

File B

```
#include <cstdio>
int main() {
int N;
scanf("%d", &N);
int acc = 0;
int step = 1;
int unused = 999;
(void)unused;
int k = 1;
while (!(k > N)) {
acc = acc + k;
k = k + step;
}
printf("%d\n", acc);
return 0;
}
```

4.2 Cặp không clone khó: P3_nonclone

Cùng khung đọc mảng bằng `for` rồi in một số, nhưng một bài tìm `max` còn bài kia tính tổng. M1 (0,000), M2 (0,005) và M5 (0,000) đều dưới ngưỡng nên nói đúng là không clone; ngược lại M3 (0,852) và M4 (0,973) vượt ngưỡng nên nói sai. MOSS cho 0,000 nên cũng đúng ở cặp này. Qua đây thấy mấy model cấu trúc (ASTNN, CPG-GNN) dễ bị khung code giống nhau đánh lừa hơn mấy model transformer.

CodeBERT — 0,000 (không clone)

File A (p3_base_max)

```
#include <bits/stdc++.h>
using namespace std;
int main() {
int n;
cin >> n;
int mx = INT_MIN, x;
for (int i = 0; i < n; i++) {
cin >> x;
if (x > mx) mx = x;
}
cout << mx << endl;
return 0;
}
```

File B (p3_n_sum)

```
#include <bits/stdc++.h>
using namespace std;
int main() {
int n;
cin >> n;
long long total = 0, x;
for (int i = 0; i < n; i++) {
cin >> x;
total += x;
}
cout << total << endl;
return 0;
}
```

GraphCodeBERT — 0,005 (không clone)

File A (p3_base_max)

```
#include <bits/stdc++.h>
using namespace std;
int main() {
int n;
cin >> n;
int mx = INT_MIN, x;
for (int i = 0; i < n; i++) {
cin >> x;
if (x > mx) mx = x;
}
cout << mx << endl;
return 0;
}
```

File B (p3_n_sum)

```
#include <bits/stdc++.h>
using namespace std;
int main() {
int n;
cin >> n;
long long total = 0, x;
for (int i = 0; i < n; i++) {
cin >> x;
total += x;
}
cout << total << endl;
return 0;
}
```

ASTNN — 0,852 (clone)

File A (p3_base_max)

```
#include <bits/stdc++.h>
using namespace std;
int main() {
int n;
cin >> n;
int mx = INT_MIN, x;
for (int i = 0; i < n; i++) {
 cin >> x;
 if (x > mx) mx = x;
 }
cout << mx << endl;
return 0;
}
```

File B (p3_n_sum)

```
#include <bits/stdc++.h>
using namespace std;
int main() {
int n;
cin >> n;
long long total = 0, x;
for (int i = 0; i < n; i++) {
 cin >> x;
 total += x;
 }
cout << total << endl;
return 0;
}
```

CPG-GNN — 0,973 (clone)

File A (p3_base_max)

```
#include <bits/stdc++.h>
using namespace std;
int main() {
 int n;
 cin >> n;
 int mx = INT_MIN, x;
 for (int i = 0; i < n; i++) {
 cin >> x;
if (x > mx) mx = x;
}
 cout << mx << endl;
 return 0;
}
```

File B (p3_n_sum)

```
#include <bits/stdc++.h>
using namespace std;
int main() {
 int n;
 cin >> n;
 long long total = 0, x;
 for (int i = 0; i < n; i++) {
 cin >> x;
total += x;
}
 cout << total << endl;
 return 0;
}
```

Hybrid — 0,000 (không clone)

File A (p3_base_max)

```
#include <bits/stdc++.h>
using namespace std;
int main() {
int n;
cin >> n;
int mx = INT_MIN, x;
for (int i = 0; i < n; i++) {
cin >> x;
if (x > mx) mx = x;
}
cout << mx << endl;
return 0;
}
```

File B (p3_n_sum)

```
#include <bits/stdc++.h>
using namespace std;
int main() {
int n;
cin >> n;
long long total = 0, x;
for (int i = 0; i < n; i++) {
cin >> x;
total += x;
}
cout << total << endl;
return 0;
}
```

MOSS (server Stanford) — 0,000 (không clone)

File A

```
#include <bits/stdc++.h>
using namespace std;
int main() {
int n;
cin >> n;
int mx = INT_MIN, x;
for (int i = 0; i < n; i++) {
cin >> x;
if (x > mx) mx = x;
}
cout << mx << endl;
return 0;
}
```

File B

```
#include <bits/stdc++.h>
using namespace std;
int main() {
int n;
cin >> n;
long long total = 0, x;
for (int i = 0; i < n; i++) {
cin >> x;
total += x;
}
cout << total << endl;
return 0;
}
```

4.3 Cặp bẫy skeleton: P1_nonclone_fact (cả 5 model đều sai)

Cũng là khung `for` 1..n rồi in một số, nhưng một bài tính tổng còn bài kia tính giai thừa. Lần này cả 5 model đều nói sai là clone: M1 0,999, M2 0,991, M3 0,960, M4 0,971, M5 0,723. MOSS cho 0,000 nên đúng ở cặp này, nhưng đúng kiểu mù (giống cặp ở mục 4.1) chứ không phải hiểu code. Cặp `p4_n_square` (số nguyên tố với số chính phương, cùng khung `for` trả lời YES/NO) cũng bị sai y hệt nên không vẽ lại hình. Chiều ngược lại thì không có cặp nào: không có cặp nào GT là clone mà cả 5 model lại bảo không clone, cả 16 cặp clone

đều bắt đúng. So với bản 256, việc chuyển M1/M2/M5 sang bản 512 sửa thêm 2 chỗ sai nữa trên bộ test (M2 và M5 ở cặp non-clone `P2_nonclone_digitsum`: 0,505 → 0,001 và 0,981 → 0,000) và không làm hỏng cặp nào.

CodeBERT — 0,999 (clone)

File A (p1_base_sum)

```
#include <bits/stdc++.h>
using namespace std;
int main() {
int n, sum = 0;
cin >> n;
for (int i = 1; i <= n; i++) {
sum += i;
}
cout << sum << endl;
return 0;
}
```

File B (p1_n_fact)

```
#include <bits/stdc++.h>
using namespace std;
int main () {
int n;
cin >> n;
long long prod = 1;
for (int i = 1; i <= n; i++) {
prod *= i;
}
cout << prod << endl;
return 0;
}
```

GraphCodeBERT — 0,991 (clone)

File A (p1_base_sum)

```
#include <bits/stdc++.h>
using namespace std ;
int main () {
int n, sum = 0;
cin >> n;
for (int i = 1; i <= n ; i++) {
sum += i;
}
cout << sum << endl ;
return 0;
}
```

File B (p1_n_fact)

```
#include <bits/stdc++.h>
using namespace std ;
int main () {
int n;
cin >> n;
long long prod = 1;
for (int i = 1 ; i <= n ; i++) {
prod *= i;
}
cout << prod << endl ;
return 0;
}
```

ASTNN — 0,960 (clone)

File A (p1_base_sum)

```
#include <bits/stdc++.h>
using namespace std;
int main() {
int n, sum = 0 ;
cin >> n;
for ( int i = 1; i <= n; i++) {
  sum += i;
 }
cout << sum << endl;
return 0;
}
```

File B (p1_n_fact)

```
#include <bits/stdc++.h>
using namespace std;
int main() {
int n;
cin >> n;
long long prod = 1 ;
for ( int i = 1; i <= n; i++) {
  prod *= i;
 }
cout << prod << endl;
return 0;
}
```

CPG-GNN — 0,971 (clone)

File A (p1_base_sum)

```
#include <bits/stdc++.h>
using namespace std;
int main() {
 int n, sum = 0;
 cin >> n;
 for (int i = 1; i <= n; i++) {
 sum += i;
}
 cout << sum << endl;
 return 0;
}
```

File B (p1_n_fact)

```
#include <bits/stdc++.h>
using namespace std;
int main() {
 int n;
 cin >> n;
 long long prod = 1;
 for (int i = 1; i <= n; i++) {
 prod *= i;
}
 cout << prod << endl;
 return 0;
}
```

Hybrid — 0,723 (clone)

File A (p1_base_sum)

```
#include <bits/stdc++.h>
using namespace std;
int main() {
int n, sum = 0;
cin >> n;
for (int i = 1; i <= n; i++) {
sum += i;
}
cout << sum << endl;
return 0;
}
```

File B (p1_n_fact)

```
#include <bits/stdc++.h>
using namespace std;
int main() {
int n;
cin >> n;
long long prod = 1;
for (int i = 1; i <= n; i++) {
prod *= i;
}
cout << prod << endl;
return 0;
}
```

MOSS (server Stanford) — 0,000 (không clone)

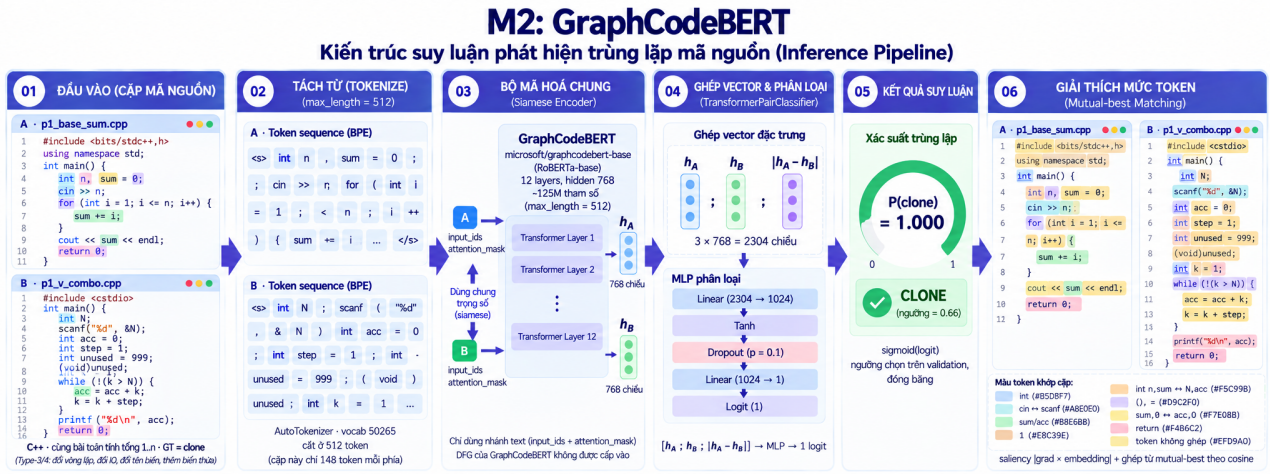
File A

```
#include <bits/stdc++.h>
using namespace std;
int main() {
int n, sum = 0;
cin >> n;
for (int i = 1; i <= n; i++) {
sum += i;
}
cout << sum << endl;
return 0;
}
```

File B

```
#include <bits/stdc++.h>
using namespace std;
int main() {
int n;
cin >> n;
long long prod = 1;
for (int i = 1; i <= n; i++) {
prod *= i;
}
cout << prod << endl;
return 0;
}
```

5 Kiến trúc phương pháp chính dự kiến (GraphCodeBERT)



6 Hạn chế

7 Kế hoạch tiếp theo